ESP32-S3
RUST LABORATORY

Practical tutorial with ESP-HAL 1.1.1

GPIO • ADC • PWM • Timers • Interrupts • UART • I²C • SPI • DMA
Watchdog • RMT • PCNT • MCPWM • TWAI • Wi-Fi • BLE • USB • Embassy

Target: ESP32-S3 / Xtensa LX7
Environment examples: Linux Mint and Rust embedded toolchain
HAL version pinned: 1.1.1

How to use this document

This document organizes the ESP32-S3 laboratory as a progressive tutorial. It explains the hardware concepts, the main ESP-HAL types and methods used in the examples, wiring, build workflow, and common integration hazards.

Important scope note: ESP-HAL is a large hardware abstraction crate. This is not a generated reference manual for every public item in the crate. It documents the functions, methods, types, and patterns used by the lab examples, and gives a map of additional peripherals. Some advanced drivers are explicitly unstable or depend on companion crates; they are not represented as guaranteed drop-in examples.

Version and compatibility discipline

- Pin esp-hal to exactly 1.1.1 while learning. Do not copy examples from the latest documentation without checking API differences.
- Use the ESP32-S3-specific documentation, not the ESP32, ESP32-C3, or generic pages.
- Keep the Cargo.toml, .cargo/config.toml, build.rs, linker configuration, and runtime settings generated by esp-generate.
- The examples below are teaching patterns. Some advanced APIs are marked unstable and may need small adjustments to match the exact 1.1.1 generated template.
- ESP32-S3 uses a dual-core Xtensa LX7 processor. It is not an ARM Cortex-M target.

Official version-specific documentation:

- ESP-HAL 1.1.1 – ESP32-S3 API
- ESP-HAL source tag v1.1.1
- The Embedded Rust Book
- esp-rs book

1. Laboratory architecture

A Hardware Abstraction Layer (HAL) exposes peripherals through Rust types and methods instead of requiring the application to manipulate peripheral registers directly. The application still needs to understand pin multiplexing, electrical limits, clocks, timing, bus protocols, interrupts, and memory constraints.

Layer	Responsibility	Examples
Application	Product logic and state machines	sensor monitor, RFID reader, camera classifier
Drivers / protocols	Device-specific transactions	LCD1602, RC522, encoder, motor driver
ESP-HAL	Safe peripheral access	GPIO, ADC, UART, I²C, SPI, timers
SoC hardware	Physical peripherals and pins	ESP32-S3 GPIO matrix, ADC, SPI2, UART0

1.1 Peripheral coverage map

Peripheral	Typical purpose	ESP32-S3 notes
GPIO	Digital input/output	Button, LED, chip select, control lines
ADC	Analog measurement	Sensor voltage; calibration and attenuation matter

LEDC	PWM generation	LED dimming, servo signal, buzzer tone
GPTimer / TIMG	Timing and periodic work	Use a hardware timer when timing precision matters
Interrupts	React to hardware events	Keep handlers short; defer work to main/task context
UART	Asynchronous serial	Console, GPS, modem, serial sensors
I²C	Two-wire bus	LCD backpack, environmental sensors
SPI	Synchronous serial	RC522, displays, flash peripherals
GDMA	Peripheral data movement	High-throughput transfers; buffer constraints apply
WDT	Fault recovery	Feed only after meaningful progress
RMT	Precise pulse trains	IR, WS2812-like signaling, timing protocols
PCNT	Pulse counting	Encoders, tachometers, frequency measurement
MCPWM	Motor-control PWM	Complementary outputs, dead time, capture
TWAI	CAN controller	Requires an external CAN transceiver
Wi-Fi / BLE	Wireless connectivity	Use compatible radio/network crates
USB	Host/device connectivity	Distinguish USB Serial/JTAG from USB OTG
Embassy	Async execution	Needs executor, time driver, and compatible HAL support

2. Toolchain installation on Linux Mint

Install system dependencies:

```
sudo apt update
sudo apt install -y curl git build-essential pkg-config libudev-dev libusb-1.0-0-dev
```

Install Rust using rustup:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
rustc --version
cargo --version
```

ESP32-S3 is an Xtensa target. Install the Espressif Rust toolchain and export its environment. The exact shell file path may be printed by espup; source the file that was actually created.

```
cargo install espup --locked
espup install
# Common export command on Linux:
source "$HOME/export-esp.sh"

rustc --version
rustup show
```

Install flashing and project-generation tools:

```
cargo install espflash --locked
cargo install esp-generate --locked
```

Generate a project for ESP32-S3. The generator's options evolve; use its help/list-options output if an option name differs:

```
esp-generate --help
esp-generate list-options
# Interactive project creation:
esp-generate
```

In the generator, choose ESP32-S3 and a bare-metal/no_std template suitable for the desired lab. If using the CLI, follow the syntax shown by the installed generator version. Keep every generated support file.

3. Project structure and Cargo configuration

```
esp32s3-rust-lab/
├── .cargo/
│   └── config.toml
├── src/
│   └── main.rs
├── examples/
│   ├── 01_gpio_output.rs
│   ├── 02_gpio_input.rs
│   ├── 03_clock.rs
│   ├── 04_adc.rs
│   ├── 05_pwm_led.rs
│   ├── 06_uart.rs
│   ├── 07_i2c.rs
│   └── 08_spi.rs
├── Cargo.toml
├── build.rs
└── rust-toolchain.toml
```

Cargo dependency baseline (illustrative; preserve the generated project's additional dependencies and features):

```
[dependencies]
esp-hal = { version = "=1.1.1", features = ["esp32s3"] }
embedded-hal = "1.0"
embedded-io = "0.6"
nb = "1.1"
critical-section = "1.2"
```

Do not replace the generated manifest with this short fragment blindly. A working ESP32-S3 project may also need runtime, linker, panic, logging, configuration, and build dependencies. In particular, the correct Xtensa target and runner are normally configured by the generated project.

3.1 Why the generated files matter

- Cargo.toml selects the HAL version, chip feature, and optional driver features.
- .cargo/config.toml selects the target triple, linker/runner, and build flags.
- build.rs may generate chip configuration and linker information.
- rust-toolchain.toml selects the Espressif-compatible compiler toolchain.
- The `unstable` feature is not a general performance switch: it exposes APIs that may change without semver guarantees.

4. Common firmware skeleton

```
#![no_std]
#![no_main]

use esp_hal::main;

#[panic_handler]
fn panic(_info: &core::panic::PanicInfo) -> ! {
    esp_hal::system::software_reset()
}

#[main]
fn main() -> ! {
    let peripherals = esp_hal::init(esp_hal::Config::default());

    // Application initialization and loop go here.
    let _ = peripherals;

    loop {
        // Main foreground loop.
    }
}
```

4.1 Explanation of the core symbols

Symbol	Meaning
#![no_std]	Compile without the standard library; typical for bare-metal firmware.
#![no_main]	Do not use the normal desktop Rust main entry point.
#[main]	ESP-HAL entry-point macro for the firmware main function.
esp_hal::init(...)	Initializes the HAL and returns the peripheral singleton collection.
Config::default()	Starts from the HAL's default system configuration.
Peripherals	Owns unique peripheral tokens; move each token into its driver once.
#[panic_handler]	Defines what the firmware does when a panic occurs.
software_reset()	Requests a chip software reset; it is a simple panic policy, not a diagnostic logger.

Ownership is important: a peripheral token should not be duplicated. Once GPIO4 is moved into a PWM channel, the same token cannot also be moved into an unrelated driver.

5. GPIO output – LED blink

Wiring: GPIO2 → series resistor (typically 220–1,000 Ω) → LED anode; LED cathode → GND. Verify the board's onboard LED wiring before using GPIO2.

```
#![no_std]
#![no_main]

use esp_hal::{
    delay::Delay,
    gpio::{Level, Output, OutputConfig},
    main,
};

#[panic_handler]
fn panic(_: &core::panic::PanicInfo) -> ! {
    esp_hal::system::software_reset()
}

#[main]
fn main() -> ! {
    let p = esp_hal::init(esp_hal::Config::default());

    let mut led = Output::new(p.GPIO2, Level::Low, OutputConfig::default());
    let delay = Delay::new();

    loop {
        led.toggle();
        delay.delay_millis(500);
    }
}
```

5.1 GPIO types and methods

API	Purpose
gpio::Output	Owns a GPIO configured as a digital output.
Output::new(pin, level, config)	Creates an output driver, sets its initial level and electrical configuration.
Level::Low / Level::High	Initial output logic level.
OutputConfig::default()	Default output configuration; use explicit drive/pull settings when required.
toggle()	Inverts the current output level.
set_high() / set_low()	Drive the output to a known logic level.
Delay::new()	Creates a blocking delay provider.
delay_millis(ms)	Busy-waits/delays for the requested milliseconds; not suitable for long delays in async systems.

A GPIO output is a logic signal, not a power supply. Do not drive motors, relays, or high-current loads directly from a GPIO pin.

6. GPIO input – push button

Example wiring: button between GPIO0 and GND. Enable an internal pull-up; the input reads LOW while pressed. Check boot-strapping restrictions for GPIO0 on your board.

```
use esp_hal::gpio::{Input, InputConfig, Pull};

let button = Input::new(
    p.GPIO0,
    InputConfig::default().with_pull(Pull::Up),
);

loop {
    if button.is_low() {
        // Button is pressed.
    } else {
        // Button is released.
    }
}
```

6.1 Input API

API	Purpose
gpio::Input	Digital input driver.
Input::new(pin, config)	Configures a pin as input.
InputConfig::default()	Default input configuration.
with_pull(Pull::Up)	Enables an internal pull-up resistor.
with_pull(Pull::Down)	Enables an internal pull-down resistor.
is_low() / is_high()	Reads the current digital logic level.

Mechanical buttons bounce. For reliable state transitions, debounce in software or with suitable hardware. Avoid treating every edge as a separate intentional press.

7. CPU clock configuration

```
use esp_hal::clock::CpuClock;
use esp_hal::Config;

let config = Config::default().with_cpu_clock(CpuClock::_80MHz);
let p = esp_hal::init(config);
```

CpuClock selects the CPU clock frequency supported by the target. Lower clocks can reduce power consumption but may reduce processing throughput. Peripheral clocks may be derived separately; do not assume every peripheral runs at the CPU clock.

8. ADC – analog sensor measurement

Example: potentiometer wiper or analog sensor output to GPIO1/ADC1 input. Connect sensor ground to board GND. Keep the ADC input within the ESP32-S3 input limits; never apply 5 V directly.

```
use esp_hal::analog::adc::{Adc, AdcConfig, Attenuation};
use nb::block;

let mut adc_config = AdcConfig::new();
let mut adc_pin = adc_config.enable_pin(p.GPIO1, Attenuation::_11dB);
let mut adc = Adc::new(p.ADC1, adc_config);

loop {
    let raw: u16 = block!(adc.read_oneshot(&mut adc_pin)).unwrap();

    // This is only a rough illustrative conversion, not calibrated voltage:
    let approximate_voltage = (raw as f32) * 3.3 / 4095.0;
    let _ = approximate_voltage;
}
```

8.1 ADC API explained

API	Purpose
AdcConfig::new()	Creates ADC configuration.
enable_pin(pin, attenuation)	Enables a supported GPIO as an ADC channel.
Attenuation::_11dB	Selects input attenuation; range and accuracy depend on the chip and calibration.
Adc::new(ADC1, config)	Creates the ADC driver for ADC1.
read_oneshot(&mut pin)	Performs one blocking-style one-shot sample through the nonblocking result interface.
nb::block!(...)	Waits until an nb::Result operation completes; can block the foreground code.

Do not treat raw ADC counts as an accurate voltage without calibration. ADC reference variation, attenuation, nonlinearity, source impedance, and board design affect results. For precision work, use the calibration support provided by the version-specific HAL or an external ADC.

9. PWM with LEDC – LED brightness

LEDC is a hardware PWM peripheral. It generates a periodic digital waveform with a configurable frequency and duty cycle. For an LED, use a series resistor. For a servo, use a separate suitable power supply and common ground.

```
// API pattern for the LEDC driver (unstable in this HAL release family).
use esp_hal::{
    ledc::{
        channel,
        timer,
        LSGlobalClkSource,
        Ledc,
        LowSpeed,
        config::Duty,
        channel::config::Config as ChannelConfig,
        timer::config::Config as TimerConfig,
    },
    gpio::DriveMode,
    time::Rate,
};

let mut ledc = Ledc::new(p.LEDC);
ledc.set_global_slow_clock(LSGlobalClkSource::APBClk);

let mut pwm_timer = ledc.timer::<LowSpeed>(timer::Number::Timer0);
pwm_timer.configure(TimerConfig {
    duty: Duty::Duty8Bit,
    clock_source: timer::LSClockSource::APBClk,
    frequency: Rate::from_khz(5),
}).unwrap();

let mut pwm = ledc.channel(channel::Number::Channel0, p.GPIO4);
pwm.configure(ChannelConfig {
    timer: &pwm_timer,
    duty_pct: 50,
    drive_mode: DriveMode::PushPull,
}).unwrap();
```

9.1 LEDC concepts and API

API / concept	Purpose
Ledc::new(LEDC)	Creates the LEDC peripheral driver.
set_global_slow_clock(...)	Selects the clock source for low-speed LEDC timers.
timer::<LowSpeed>(Timer0)	Selects a low-speed PWM timer.
Duty::Duty8Bit	Selects an 8-bit duty resolution (0–255 levels).
Rate::from_khz(5)	Represents a frequency of 5 kHz.
timer.configure(...)	Configures PWM timer frequency, clock source and duty resolution.
channel(Channel0, GPIO)	Associates a PWM channel with an output pin.
channel.configure(...)	Connects the channel to the timer and configures duty and drive mode.
DriveMode::PushPull	Configures a push-pull output stage.

The LEDC module and its configuration APIs are version-sensitive/unstable in the 1.1.x family. If the compiler reports an import or type mismatch, use the exact ESP-HAL 1.1.1 LEDC module documentation and the generated example for that tag rather than changing the HAL version casually.

9.2 Servo PWM principle

- A common hobby servo expects a pulse approximately every 20 ms (about 50 Hz).

- Pulse widths near 1.0 ms, 1.5 ms, and 2.0 ms often correspond roughly to one end, center, and the other end.
- The exact safe pulse range varies by servo; begin conservatively and do not force mechanical end stops.
- Power the servo from a suitable external supply. Join the supply ground and ESP32-S3 ground.

10. Hardware timers

Use a hardware timer when an application needs a periodic event or accurate timing independent of loop execution. For simple teaching delays, Delay is easier; for periodic sampling or scheduling, use a timer or async timer driver.

```
// Illustrative timer-group pattern; check the exact 1.1.1 timer API.
use esp_hal::{
    timer::timg::TimerGroup,
    time::Duration,
};

let timg0 = TimerGroup::new(p.TIMG0);
let mut timer = esp_hal::timer::OneShotTimer::new(timg0.timer0);
timer.delay_millis(1000);
```

The precise timer type paths and start/wait methods can differ between stable and unstable APIs. The 1.1.1 documentation is authoritative for the exact constructor and timer trait in your project.

11. GPIO interrupts

An interrupt handler runs when a configured hardware event occurs. The handler should acknowledge or clear the event as required, set a flag, and return quickly. Do not perform long delays, formatted logging, blocking bus transfers, or complex application logic inside an ISR.

```
// Conceptual pattern: configure a GPIO for a falling-edge event,
// set an atomic flag in the interrupt handler, and process it in main.
//
// The exact handler attribute, interrupt binding, and event API must
// follow the ESP-HAL 1.1.1 interrupt example for the selected GPIO.

use core::sync::atomic::{AtomicBool, Ordering};

static BUTTON_EVENT: AtomicBool = AtomicBool::new(false);

// In the GPIO interrupt handler:
fn signal_button_event() {
    BUTTON_EVENT.store(true, Ordering::Relaxed);
}

// In the main loop:
loop {
    if BUTTON_EVENT.swap(false, Ordering::Relaxed) {
        // Handle the button event outside the ISR.
    }
}
```

The atomic flag is a safe communication pattern for a simple event. For shared peripheral state, use the critical-section mechanism and the exact interrupt registration API supported by ESP-HAL 1.1.1. Re-arm or clear edge detection according to the driver API and debounce the physical button.

12. UART – serial communication

Example wiring for UART1: ESP TX GPIO43 → peripheral RX; ESP RX GPIO44 ← peripheral TX; common GND. Logic levels must be 3.3 V compatible.

```
use esp_hal::{
    uart::{Config, Uart},
    time::Rate,
};
use embedded_io::{Read, Write};

let mut uart = Uart::new(
    p.UART1,
    Config::default().with_baudrate(115_200),
).unwrap()
 .with_tx(p.GPIO43)
 .with_rx(p.GPIO44);

let mut rx = [0u8; 64];

loop {
    match uart.read(&mut rx) {
        Ok(n) if n > 0 => {
            let _ = uart.write_all(&rx[..n]);
        }
        Ok(_) => {}
        Err(_) => {}
    }
}
```

12.1 UART API

API	Purpose
Uart::new(UART1, Config)	Creates a UART driver on the selected UART peripheral.
Config::default()	Starts with default UART configuration.
with_baudrate(115_200)	Sets the serial bit rate.
with_tx(GPIO)	Routes UART TX to a GPIO pin.
with_rx(GPIO)	Routes UART RX to a GPIO pin.
Read::read()	Reads available bytes through embedded-io.
Write::write_all()	Writes all bytes or returns an error.

The embedded_io traits are imported to make read/write methods available. For a robust protocol, add framing, timeouts, buffer management, error handling, and a parser. Do not assume one read call returns a complete message.

13. I²C – LCD and sensors

Example wiring matching the user's LCD setup: SDA GPIO41, SCL GPIO40, GND to GND. Verify that the LCD backpack pull-ups do not pull the ESP32-S3 pins to 5 V.

```
use esp_hal::{
    i2c::master::{Config, I2c},
    time::Rate,
};

let mut i2c = I2c::new(
    p.I2C0,
    Config::default().with_frequency(Rate::from_khz(100)),
).unwrap()
 .with_sda(p.GPIO41)
 .with_scl(p.GPIO40);

let mut response = [0u8; 2];
let result = i2c.write_read(0x27, &[0x00], &mut response);
```

13.1 I²C API and protocol

API	Purpose
I2c::new(I2C0, Config)	Creates an I²C master driver.
with_frequency(Rate::from_khz(100))	Sets the bus clock to 100 kHz.
with_sda(GPIO)	Routes SDA to the selected GPIO.
with_scl(GPIO)	Routes SCL to the selected GPIO.
write_read(address, write, read)	Writes a command/register address, then reads response bytes.

The 0x27 address is common for LCD backpacks but is not universal. A successful I²C transaction does not by itself implement the LCD protocol. To control an LCD1602, use a PCF8574-compatible LCD driver or implement the expander nibble/enable/backlight sequence.

I²C electrical requirements:

- SDA and SCL are open-drain lines and need pull-up resistors.
- Pull-ups must go to a voltage safe for the ESP32-S3 (normally 3.3 V).
- All devices on the bus need a common ground.
- Check address conflicts and total bus capacitance when multiple modules are connected.

14. SPI – RC522 RFID reader

Typical SPI wiring: SCK GPIO12, MOSI GPIO11, MISO GPIO13, CS/SDA GPIO10, RST GPIO9, 3.3 V, GND. On an RC522 module, the pin marked SDA is usually SPI chip-select, not I²C SDA.

```
use esp_hal::{
    spi::master::{Config, Spi},
    time::Rate,
};
use embedded_hal::spi::SpiBus;

let mut spi = Spi::new(
    p.SPI2,
    Config::default()
        .with_frequency(Rate::from_mhz(1))
        .with_mode(embedded_hal::spi::MODE_0),
).unwrap()
 .with_sck(p.GPIO12)
 .with_mosi(p.GPIO11)
 .with_miso(p.GPIO13);

let mut cs = esp_hal::gpio::Output::new(
    p.GPIO10,
    esp_hal::gpio::Level::High,
    esp_hal::gpio::OutputConfig::default(),
);

cs.set_low();
spi.write(&[0xAA, 0x55]).unwrap();
cs.set_high();
```

14.1 SPI API

API	Purpose
Spi::new(SPI2, Config)	Creates an SPI master driver.
with_frequency(Rate)	Sets the SPI clock frequency.
with_mode(MODE_0)	Selects clock polarity/phase mode.
with_sck(GPIO)	Routes SCK to a GPIO.
with_mosi(GPIO)	Routes controller-to-peripheral data.
with_miso(GPIO)	Routes peripheral-to-controller data.
SpiBus::write()	Transmits a byte buffer.
Output::set_low()/set_high()	Manually asserts/deasserts chip select.

The example is a bus-level demonstration, not an RC522 driver. RFID register addresses, command framing, CRC, reset timing, card anticollision, and UID decoding must be implemented by a dedicated driver or application code.

15. DMA – Direct Memory Access

DMA transfers data between memory and a peripheral without requiring the CPU to copy every byte. It is useful for high-throughput SPI, I²S, camera and other data paths. DMA buffers must satisfy the driver's alignment, size, lifetime, and memory-region requirements.

```
// Conceptual setup pattern from the DMA-enabled SPI API family.
// Check the exact 1.1.1 constructor signatures and buffer constraints.

let (rx_buffer, rx_descriptors, tx_buffer, tx_descriptors) = dma_buffers!(32000);

let rx = DmaRxBuf::new(rx_descriptors, rx_buffer).unwrap();
let tx = DmaTxBuf::new(tx_descriptors, tx_buffer).unwrap();

// Configure the SPI peripheral with a DMA channel and attach the buffers.
// A transfer must still be started and completed using the selected driver's API.
```

This section intentionally does not pretend that buffer construction alone performs a DMA transfer. The complete transfer API depends on the chosen peripheral driver and ESP-HAL release. Camera capture on ESP32-S3 uses the camera/LCD_CAM path and is not equivalent to attaching SPI DMA buffers.

16. Watchdog timer

A watchdog detects firmware that stops making progress. The application must feed it only after critical work has completed; feeding it unconditionally from a tight loop can hide deadlocks.

```
// Illustrative TimerGroup watchdog setup.
// Check the exact watchdog API and timeout units in ESP-HAL 1.1.1.

use esp_hal::timer::timg::TimerGroup;
use esp_hal::time::Duration;

let mut timers = TimerGroup::new(p.TIMG0);
timers.wdt.set_timeout(Duration::from_secs(5));
timers.wdt.enable();

// Feed only after a successful application progress checkpoint:
timers.wdt.feed();
```

Design rule: define what counts as progress (for example, sensor acquisition + control update + task heartbeat) before feeding the watchdog.

17. Advanced peripherals – what they do

17.1 RMT (Remote Control)

RMT generates and/or receives timed pulse sequences. It is useful for IR remote protocols, addressable LEDs, and custom one-wire timing. Configure a channel, clock resolution, pulse durations and levels, then transmit or receive a sequence. RMT APIs are version-sensitive; use the 1.1.1 RMT example.

17.2 PCNT (Pulse Counter)

PCNT counts input edges in hardware and can apply filtering/limits. For a tachometer, if ΔN pulses are counted over Δt seconds and the sensor produces PPR pulses per revolution:

$$RPM = 60 \times \Delta N / (PPR \times \Delta t)$$

Confirm whether PPR counts one edge, both edges, or quadrature transitions; otherwise the RPM scale will be wrong.

17.3 MCPWM

MCPWM is intended for motor-control waveforms and provides more advanced timing features than basic LED PWM, including synchronized outputs and dead-time support on suitable channels. Use a motor driver or gate driver; never connect a motor directly to an ESP32-S3 GPIO.

17.4 TWAI (CAN)

TWAI is the ESP32-S3 CAN controller. It needs an external physical-layer transceiver (for example, a suitable 3.3 V CAN transceiver). The controller's TX/RX pins are not CANH/CANL. Termination and bus topology must follow CAN rules.

17.5 Wi-Fi

ESP-HAL is the hardware layer; Wi-Fi networking requires the Espressif radio integration and a compatible network stack. A typical architecture is esp-hal + esp-radio + embassy-net (or another supported stack). Pin compatible versions together; do not assume the newest radio crate is compatible with esp-hal 1.1.1.

17.6 Bluetooth Low Energy

BLE requires a compatible controller/host stack and radio integration. A Rust BLE host such as Trouble may be relevant, but compatibility with the chosen ESP radio backend and HAL release must be verified. ESP32-S3 supports BLE, not Bluetooth Classic.

17.7 USB

Distinguish the USB Serial/JTAG function from the USB OTG controller. The board may also include a separate USB-to-UART bridge. Connector wiring and boot mode depend on the development board. USB OTG support uses additional driver dependencies and configuration.

17.8 Embassy async

Embassy provides an async executor and async peripheral ecosystem. An async task is not a complete application by itself: you need an executor, task spawning, time driver, and async-capable drivers. Do not mix Embassy examples from incompatible HAL versions.

18. Pin planning and wiring

The following is a suggested lab pin plan for separate exercises. It is not safe to connect every peripheral at once. The user's OV2640 camera wiring consumes many of these GPIOs.

Function	Suggested GPIO	Notes
LED output	GPIO2	Use a series resistor; check board-specific LED.
Button input	GPIO0	Boot-strapping pin; use carefully.
ADC input	GPIO1	Analog-capable pin; stay within voltage range.
PWM output	GPIO4	Conflicts with the user's OV2640 SIOD pin.
UART1 TX/RX	GPIO43 / GPIO44	Check USB/UART board routing.
I²C SDA/SCL	GPIO41 / GPIO40	Matches the user's LCD setup.
SPI SCK/MOSI/MISO	GPIO12 / GPIO11 / GPIO13	Check camera pin conflicts.
SPI CS / reset	GPIO10 / GPIO9	RC522 example; conflicts with camera data pins.

18.1 Existing OV2640 camera pinout – conflict warning

Camera signal	GPIO

XCLK	15
SIOD	4
SIOC	5
D0	11
D1	9
D2	8
D3	10
D4	12
D5	18
D6	17
D7	16
VSYNC	6
HREF	7
PCLK	13

Because the OV2640 uses GPIO4–13 and several other pins, the GPIO4 PWM, GPIO9/10/11/12/13 SPI/RC522, and GPIO5–7 advanced-I/O suggestions cannot be used simultaneously with that camera wiring. Create a pin map for the exact ESP32-S3 module/board before wiring. Also check flash/PSRAM and strapping pin restrictions.

19. Build, flash, monitor

```
# From the project directory, after sourcing the ESP toolchain:
source "$HOME/export-esp.sh"

# Check and build:
cargo check
cargo build --release

# Run/flash using the runner configured by esp-generate:
cargo run --release

# If the project includes examples:
cargo run --release --example 01_gpio_output

# Inspect the connected board:
espflash board-info

# Open serial monitor (select the correct serial port if needed):
espflash monitor
```

The exact cargo run behavior depends on the generated .cargo/config.toml. Some templates use espflash as the runner; others may require an explicit port or a separate flash command. Use `espflash --help` and the generated project README.

19.1 Troubleshooting checklist

Symptom	Checks
`link.exe` or linker not found	On Linux, verify Xtensa toolchain is active and `source ~/export-esp.sh` was run. On Windows, install/configure the required linker toolchain.
Wrong target / unsupported target	Check rust-toolchain.toml and .cargo/config.toml; ESP32-

	S3 is Xtensa.
HAL item not found	Confirm esp-hal is exactly 1.1.1, the ESP32-S3 feature is enabled, and the import path matches that version.
Unstable API error	Check whether the generated project enables the required unstable feature and whether the API is available for ESP32-S3.
Flash fails	Check cable, port, boot mode, USB connector, permissions, and board reset procedure.
No serial output	Check monitor port, baud rate, USB Serial/JTAG vs USB-UART bridge, and logging configuration.
I²C device not responding	Check address, SDA/SCL wiring, pull-up voltage, common GND, and device power.
ADC readings wrong	Check input voltage, attenuation, calibration, source impedance, and pin capability.
Camera no longer works	Check for GPIO conflicts and ensure camera pin mapping remains unchanged.

20. Suggested progression

1. Compile the generated empty project before adding peripherals.
2. Blink one LED using GPIO output.
3. Read one button using GPIO input and add debounce.
4. Measure a potentiometer using ADC and compare raw values with a multimeter.
5. Dim an LED using LEDC PWM.
6. Send and receive bytes over UART.
7. Scan and communicate with a known I²C sensor; then integrate the LCD1602 driver.
8. Test SPI with a loopback or known peripheral; then implement RC522 protocol support.
9. Add a periodic timer and a short interrupt-driven event.
10. Add watchdog supervision after the application has clear progress checkpoints.
11. Explore DMA, RMT, PCNT, MCPWM and TWAI one at a time.
12. Only then integrate Wi-Fi, BLE, USB or Embassy async execution.

21. ESP-HAL API glossary – functions and methods used

Function / method	What it does	Typical use
esp_hal::init(config)	Initializes HAL and returns peripherals	Startup
Config::default()	Creates default system config	Startup
with_cpu_clock(clock)	Selects CPU frequency	Clock setup
Output::new(pin, level, config)	Configures output GPIO	LED/control
Input::new(pin, config)	Configures input GPIO	Button/sensor
with_pull(Pull::Up/Down)	Configures input pull resistor	Floating-input prevention
is_low()/is_high()	Reads input state	Button logic
set_low()/set_high()/toggle()	Changes output level	LED/control

Delay::new()	Creates blocking delay	Simple timing
delay_millis(ms)	Waits in milliseconds	Blink/demo
AdcConfig::new()	Creates ADC config	ADC
enable_pin(pin, attenuation)	Enables ADC channel	ADC
Adc::new(unit, config)	Creates ADC driver	ADC
read_oneshot(&mut pin)	Samples analog input	ADC
Ledc::new(LEDC)	Creates LEDC driver	PWM
set_global_slow_clock(...)	Selects LEDC slow clock	PWM
timer::<LowSpeed>(...)	Selects PWM timer	PWM
configure(...)	Applies driver configuration	PWM/timer
channel(...)	Creates/selects PWM channel	PWM
Uart::new(...)	Creates UART driver	Serial
with_baudrate(...)	Sets UART baud rate	Serial
with_tx()/with_rx()	Routes UART pins	Serial
read()/write_all()	Reads/writes serial bytes via embedded-io	Serial
I2c::new(...)	Creates I²C master	I²C
with_frequency(...)	Sets I²C clock	I²C
with_sda()/with_scl()	Routes I²C pins	I²C
write_read(...)	Combined I²C write then read	Sensor register access
Spi::new(...)	Creates SPI master	SPI
with_frequency()/with_mode()	Sets SPI rate and mode	SPI
with_sck()/with_mosi()/with_miso()	Routes SPI pins	SPI
SpiBus::write()	Transmits SPI bytes	SPI
TimerGroup::new(...)	Creates timer-group driver	Timer/WDT
wdt.set_timeout()/enable()/feed()	Configures/enables/feeds watchdog	Recovery
software_reset()	Requests a software reset	Panic policy

The glossary lists the methods used or discussed in this tutorial. It is not an exhaustive listing of every ESP-HAL public method. Consult the versioned API pages for full signatures, trait bounds, feature requirements, and peripheral-specific availability.

22. Official references

- ESP-HAL 1.1.1 – ESP32-S3 API root
- ESP-HAL 1.1.1 – ADC module
- ESP-HAL 1.1.1 – UART module
- ESP-HAL source code, v1.1.1
- esp-generate project generator
- ESP-RS book
- ESP32-S3 datasheet
- Embedded Rust Book

- Embassy documentation

Final engineering notes

Treat this laboratory as a set of independent exercises. Compile after each peripheral is added, test each circuit with current-limited and voltage-safe wiring, and keep a pin allocation table for the exact development board. The camera, PSRAM, flash, USB, boot-strapping pins, and external modules can constrain which GPIOs remain available.

For a production project, add error handling, watchdog supervision, logging, power budgeting, task ownership, and driver-level tests. Record the exact board/module revision, ESP-HAL version, Rust toolchain version, and generated project configuration alongside the source code.